

02

TypeScript Refresher (Angular için)

🕒 Başlangıç · Sıfır → Junior

🔗 Bu modülde ne öğreneceğiz?

- ✓ TypeScript'in neden var olduğunu ve Angular'ın onu neden zorunlu kıldığını
- ✓ Temel tipleri ve interface'lerle nesne şekillerini tanımlamayı
- ✓ Generics ile tek bir kodu birçok tiple güvenle kullanmayı
- ✓ Decorators'ın ne olduğunu (Angular'daki @Component, @Injectable mantığı)
- ✓ Utility types (Partial, Pick, Omit, Record...) ile tipleri dönüştürmeyi
- ✓ Type guards & narrowing ile union tipleri güvenle daraltmayı

Ön koşul: Modül 1 ve temel JavaScript bilgisi. TypeScript'i daha önce hiç görmediysen sorun değil; her şeyi sıfırdan, Angular'da nasıl kullanıldığını göstererek ele alacağız. Amacımız "TypeScript uzmanı" olmak değil; **Angular'ı rahat yazacak kadar** sağlam bir temel kurmak.



Angular'ın dili TypeScript'tir. O yüzden Angular'ı rahat yazmak için önce bu dile ısınmamız gerekiyor. İyi haber: TypeScript yeni bir dil değil; **JavaScript'in üstüne "tipler" eklenmiş hâli**. Bildiğin tüm JS, TypeScript'te de geçerli.

Neden TypeScript? JavaScript'in Sorunu

Saf JavaScript'te bir değişkenin türü yoktur; her şey çalışma anında (runtime) belli olur. Bu da hataların **kullanıcı karşısında** patlaması demektir:

```
JAVASCRIPT

// Saf JavaScript
function selamla(kullanici) {
  return "Merhaba " + kullanici.ad.toUpperCase();
}

selamla({ isim: "Efsane" });
// Çöker: kullanici.ad "undefined" -> toUpperCase() hata verir.
// Ama bu hatayı ancak kod ÇALIŞINCA fark edersin.
```

TypeScript ise aynı hatayı sen daha yazarken, **editörde kırmızı çizgiyle** gösterir. “ad diye bir alan yok, sen isim demişsin” der. Yani hataları **çalışmadan önce** yakalarsın.

❗ Tek cümleyle

TypeScript = JavaScript + **derleme zamanında tip kontrolü**. Daha az runtime hatası, daha iyi otomatik tamamlama, daha okunaklı kod.

TypeScript Nasıl Çalışır?

Tarayıcı TypeScript'i doğrudan çalıştıramaz. TypeScript **derlenir** (compile) ve sade JavaScript'e dönüşür; tarayıcıya o JS gider. Önemli nokta: **tipler sadece geliştirme/derleme aşamasında vardır**, üretilen JS'te tip bilgisi kalmaz. Yani tipler senin için bir “güvenlik ağı”, çalışan koda yük getirmez. Angular'da bu derlemeyi senin yerine CLI hallediyor; ekstra bir şey kurmana gerek yok.

Temel Tipler

Bir değişkene tip vermeye **tip ataması (annotation)** denir: `degisken: tip`. En sık kullanılanlar:

```

let ad: string = "Efsane";
let yas: number = 28;
let aktifMi: boolean = true;

// Diziler
let puanlar: number[] = [90, 85, 100];
let isimler: string[] = ["Ada", "Efe"];

// Birden fazla tip olabilir (union type)
let kimlik: string | number = 42;
kimlik = "ABC-42"; // ikisi de geçerli

// Sadece belirli değerler (literal types)
let yon: "sol" | "sag" | "yukari" | "asagi" = "sol";

```

Fonksiyon dönmüyorsa dönüş tipi `void`; değeri henüz yoksa `null` veya `undefined` olabilir.

Tip Çıkarımı (Type Inference)

Her zaman tipi elle yazmak zorunda değilsin. TypeScript çoğu zaman **kendi anlar**:

```

let sehir = "İstanbul"; // TypeScript bunu otomatik "string" sayar
sehir = 42;              // HATA: number, string'e atanamaz

// 0 yüzden değişkene başlangıç değeri verdiğinde
// tipi tekrar yazmana çoğu zaman gerek kalmaz.

```

Pratik kural

Başlangıç değeri olan değişkenlerde tipi yazma (inference yeterli). Fonksiyon parametrelerinde ve nesne modellerinde ise tipleri **açıkça yaz** — orada TypeScript tahmin edemez ve hatalar oradan çıkar.

Fonksiyon Tipleri

Parametrelerin ve dönüş değerinin tipini belirtiriz:

```
function topla(a: number, b: number): number {
  return a + b;
}

// Opsiyonel parametre (?) ve varsayılan değer
function selamla(ad: string, unvan?: string): string {
  return unvan ? `${unvan} ${ad}` : ad;
}

// Ok fonksiyonu
const kareAl = (x: number): number => x * x;
```

any ve unknown

any “her şey olabilir, kontrolü kapat” demektir; TypeScript’in tüm faydasını yok eder. Mümkün olduğunca **kaçın**. Tipini gerçekten bilmediğin bir değer varsa **unknown** kullan — o, kullanmadan önce tip kontrolü yapmaya zorlar (daha güvenli).

Interface — Nesnelerin Şekli

Uygulamalar nesnelerle doludur (kullanıcı, ürün, görev...). Bir nesnenin hangi alanları ve hangi tipleri taşıdığını **interface** ile tanımlarız:

```
interface Kullanici {
  id: number;
  ad: string;
  email: string;
  yas?: number; // ? => opsiyonel (olmayabilir)
  readonly olusturulma: Date; // readonly => sonradan değiştirilemez
}

const u: Kullanici = {
  id: 1,
  ad: "Efsane",
  email: "efsane@ornek.com",
  olusturulma: new Date(),
};

u.ad = "Emre"; // OK
u.olusturulma = new Date(); // HATA: readonly
```

Interface’ler **geniştirilebilir** (kalıtım):

```
interface Yoneticici extends Kullanici {
  yetkiler: string[];
}
```

❗ interface mi, type mı?

Benzer işi `type Kullanici = { ... }` ile de yapabilirsin. Pratikte: nesne **şekilleri** için `interface`, union/birleşik tipler (`type Durum = 'acik' | 'kapali'`) için `type` tercih edilir. Angular topluluğunda model tanımlarında `interface` çok yaygındır.

Angular Bağlamında

Bir Angular component'inde tipler şöyle görünür (detayını Modül 4'te göreceğiz):

```
import { Component, signal } from '@angular/core';

interface Gorev {
  id: number;
  baslik: string;
  tamamlandi: boolean;
}

@Component({
  selector: 'app-gorevler',
  template: `...`,
})
export class Gorevler {
  // signal'in içindeki tip de TypeScript ile güvende
  gorevler = signal<Gorev[]>([]);

  ekle(baslik: string): void {
    this.gorevler.update((liste) => [
      ...liste,
      { id: Date.now(), baslik, tamamlandi: false },
    ]);
  }
}
```

🔍 Mini Quiz

1. TypeScript hataları ne zaman yakalar: çalışma anında mı, derleme zamanında mı?
2. Üretilen JavaScript'te tip bilgisi kalır mı?
3. `yas?: number` ifadesindeki `?` ne anlama gelir?
4. Neden `any` yerine `unknown` tercih edilir?

🔑 Cevaplar

1. Derleme zamanında (kod çalışmadan önce, editörde).
2. Hayır. Tipler sadece geliştirme/derleme aşamasındadır; üretilen JS'te kalmaz.
3. Alan opsiyoneldir; nesnede bulunmayabilir.
4. `unknown` kullanmadan önce tip kontrolü yapmaya zorlar; `any` ise tüm tip güvenliğini kapatır.

⊗ Anti-Patterns

- ⊗ **Her yere `any` yazmak.** TypeScript'i kullanmamakla aynı şey; tip güvenliğini yok eder.
- ⊗ **Model nesnelerini tipsiz bırakmak.** Kullanıcı/ürün/görev gibi yapıları interface ile tanımla; hatalar erken çıksın.
- ⊗ **Tip ataması ile değer atamasını karıştırmak.** `let x: number` tip; `let x = 5` değer (ve tip çıkarımı).

✅ Bu Dersten Çıkarılacaklar

- ✓ TypeScript = JavaScript + derleme zamanı tip kontrolü; hataları erken yakalar.
- ✓ Tipler sadece geliştirme aşamasında vardır; çalışan JS'e yük bindirmez.
- ✓ Temel tipler: string, number, boolean, diziler, union (`|`), literal tipler.
- ✓ Tip çıkarımı çoğu yerde tipi senin için anlar; parametre ve modellerde tipi açık yaz.
- ✓ interface, nesnelerin şeklini tanımlar (opsiyonel `?`, `readonly`, `extends`).



Generics ilk bakışta korkutucu görünür ama fikri çok basittir: **tipi sabit yazmak yerine, “parametre” olarak almak.** Böylece aynı kodu birçok tipe, üstelik tip güvenliğini kaybetmeden kullanırsın.

Problem: Aynı Mantık, Farklı Tipler

Diyelim bir dizinin ilk elemanını döndüren bir fonksiyon yazıyorsun. **any** ile yazarsan tip güvenliği kaybolur:

TYPESCRIPT

```
function ilkEleman(dizi: any[]): any {
  return dizi[0];
}

const sayi = ilkEleman([1, 2, 3]);
// sayi'nin tipi "any" -> TypeScript artık sana yardım edemez
sayi.toUpperCase(); // HATA değil sanılır ama runtime'da çöker
```

Çözüm: Generic Tip Parametresi

Tipi **<T>** ile “dışarıdan” alıyoruz (T = Type; istediğin harfi kullanabilirsin ama T gelenektir):

TYPESCRIPT

```
function ilkEleman<T>(dizi: T[]): T {
  return dizi[0];
}

const sayi = ilkEleman([1, 2, 3]); // T = number -> sayi: number
const ad = ilkEleman(["Ada", "Efe"]); // T = string -> ad: string

sayi.toFixed(2); // OK (number)
ad.toUpperCase(); // OK (string)
```

Tek fonksiyon, ama her çağrıda doğru tip korunuyor. TypeScript **T**'yi verdiğin değere göre otomatik çıkarır.

Metafor

Generic, bir **kalıp** gibidir. Aynı kek kalıbına çikolatalı da vanilyalı da dönebilirsin; kalıp aynı, içerik (tip) değişir. **<T>** işte o “içeriğin tipi”.

Generic Kısıtlama (Constraint)

Bazen “herhangi bir tip” değil, “şu özelliğe sahip tipler” istersin. **extends** ile sınırlarsın:

```
// Sadece "length" özelliği olan tipleri kabul et
function uzunlukYaz<T extends { length: number }>(deger: T): T {
  console.log(deger.length);
  return deger;
}

uzunlukYaz("merhaba"); // OK (string'in length'i var)
uzunlukYaz([1, 2, 3]); // OK (dizinin length'i var)
uzunlukYaz(42); // HATA: number'ın length'i yok
```

Generic Interface'ler

Generics sadece fonksiyonlarda değil, interface ve sınıflarda da kullanılır. Çok yaygın bir örnek: bir API cevabını saran kapsayıcı:

```
interface ApiCevap<T> {
  basarili: boolean;
  veri: T;
  mesaj?: string;
}

// Kullanıcı listesi dönen cevap
const cevap1: ApiCevap<Kullanici[]> = { basarili: true, veri: [] };

// Tek bir ürün dönen cevap
const cevap2: ApiCevap<Urun> = { basarili: true, veri: urun };
```

Angular'da Generics Her Yerde

Generics'i öğrenmen şart, çünkü Angular API'leri baştan aşağı generic:

API	Örnek	Anlamı
<code>signal<T>()</code>	<code>signal<number>(0)</code>	Değeri number olan bir signal
<code>HttpClient.get<T>()</code>	<code>http.get<Kullanici[]>(url)</code>	Cevabı Kullanici[] olarak tipler
<code>WritableSignal<T></code>	<code>WritableSignal<Gorev[]></code>	Yazılabilir signal'in tip imzası
<code>EventEmitter<T></code>	<code>new EventEmitter<string>()</code>	string yayan olay (eski output stili)

❗ Korkma

Kendi generic fonksiyonlarını yazman ilk başta nadir olacak. Ama Angular'ın generic API'lerini **okuyup doğru tip vermen** her gün gerekecek. Bu dersin amacı tam olarak o: `get<Kullanici[]>()` gördüğünde ne olduğunu anlamak.

🔍 Mini Quiz

1. Generic'in temel fikri tek cümleyle nedir?
2. `function ilk<T>(d: T[]): T` imzasında `T` ne zaman belirlenir?
3. `<T extends { length: number }>` ne işe yarar?
4. `http.get<Kullanici[]>(url)` çağrısında generic ne sağlar?

🔑 Cevaplar

1. Tipi sabitlemek yerine parametre olarak almak; aynı kodu birçok tipte güvenle kullanmak.
2. Fonksiyon çağrıldığında, verilen argümana göre TypeScript otomatik çıkarır.
3. Generic'i kısıtlar: yalnızca `length` özelliği olan tipler kabul edilir.
4. Dönen cevabın tipini `Kullanici[]` yapar; sonrasında tam tip güvenliği ve otomatik tamamlama.

❌ Anti-Patterns

- ❌ **Generic yerine `any` kullanmak.** İşi “çözer” gibi görünür ama tip güvenliğini yok eder.
- ❌ **Gereksiz yere generic yazmak.** Tip değişmiyorsa generic'e gerek yok; sadelik önemli.
- ❌ **HTTP çağrısına tip vermemek.** `http.get(url)` yerine `http.get<Kullanici[]>(url)` yaz; yoksa cevap `Object` olur.

✅ Bu Dersten Çıkarılacaklar

- ✓ Generic = tipi parametre olarak almak (`<T>`); tek kod, çok tip, tam güvenlik.
- ✓ TypeScript `T`'yi verdiği değerden otomatik çıkarır.
- ✓ `extends` ile generic'i belirli özelliklere sahip tiplerle sınırlarsın.
- ✓ Generic interface'ler API cevapları gibi kapsayıcılar için idealdir (`ApiCevap<T>`).
- ✓ Angular'da `signal<T>`, `get<T>` gibi generic'leri okuyup doğru tip vermek günlük iştir.



Decorators (Kavramsal)

Angular kodunda sürekli `@Component`, `@Injectable` gibi `@` ile başlayan ifadeler göreceksin. Bunlara **decorator** denir. Bu derste amacımız kendi decorator'ımızı yazmak değil — **ne olduklarını kavramsal olarak anlamak**, çünkü Angular'ın her yerinde karşına çıkacaklar.

Decorator Nedir?

Decorator, bir sınıfa (veya alanına/metoduna) ek bilgi ve davranış ekleyen özel bir işarettir. Teknik olarak bir **fonksiyondur**; hemen üstüne yazıldığı şeyi alır ve ona “metadata” (üstveri) ilişitir. Angular derleyicisi bu metadata'yı okuyup ne yapacağını anlar.

Metafor

Bir kutuya yapıştırdığın **etiket** gibi düşün: “Kırılır”, “Bu taraf yukarı”. Kutunun içeriğini değiştirmez ama onu taşıyan sisteme **nasıl davranacağını** söyler. `@Component` da bir sınıfa “bu bir Angular bileşenidir, şu template'i kullanır” etiketini yapıştırır.

Angular'da En Çok Göreceğin Decorator'lar

Decorator	Nereye	Ne der?
<code>@Component</code>	Sınıfa	“Bu bir bileşen; selector'ı ve template'i şu”
<code>@Injectable</code>	Sınıfa	“Bu bir servis; DI sistemi bunu enjekte edebilir”
<code>@Directive</code>	Sınıfa	“Bu bir directive (davranış ekleyen yapı)”
<code>@Pipe</code>	Sınıfa	“Bu bir pipe (template'te veri dönüştürür)”

Örneğin `@Component` bir nesne (metadata) alır:

TYPESCRIPT

```
@Component({
  selector: 'app-counter', // HTML'de <app-counter> olarak kullanılır
  template: `<p>{{ count() }}</p>`,
})
export class Counter {
  count = signal(0);
}
```

Buradaki `@Component({...})` kısmı sınıfa “Angular bileşeni” kimliğini ve yapılandırmasını ekler. Sen sadece bilgileri verirsin; gerisini Angular halleder.

Kavramsal: Bir Decorator Aslında Fonksiyondur

Çok basitleştirilmiş bir örnek (bunu yazmayacaksın, sadece fikir):

TYPESCRIPT

```
// "Bu sınıfı oluştururken konsola haber ver" diyen sahte bir decorator
function Logla(hedef: Function) {
  console.log("Bir sınıf tanımlandı:", hedef.name);
}

@Logla
class Ornek {}
// Ornek tanımlandığında Logla(Ornek) çağrılır.
```

Angular'ın decorator'ları da aynı prensiple çalışır; sadece çok daha gelişmiş metadata üretir ve bunu Angular derleyicisi kullanır.

Önemli: Modern Angular Decorator'ları Azaltıyor

Eski Angular, bileşen girdi/çıkıtları için de decorator kullanırdı: `@Input()` ve `@Output()`. Modern Angular (v17+) bunların yerine **signal-tabanlı fonksiyonlar** öneriyor:

TYPESCRIPT

```
// ESKİ stil (decorator) – hâlâ çalışır ama artık tercih edilmez
@Input() baslik: string = '';
@Output() secildi = new EventEmitter<number>();

// MODERN stil (v21) – fonksiyon tabanlı
baslik = input<string>('');
secildi = output<number>();
```

❗ Sınıf decorator'ları kalıyor

`@Component`, `@Injectable`, `@Directive`, `@Pipe` gibi **sınıf decorator'ları** Angular'ın temelidir ve devam ediyor. Azaltılan şey, sınıf içi `@Input/@Output` gibi alan decorator'larıdır. Bunların modern karşılıklarını (`input()`, `output()`, `model()`) Modül 4'te detaylı göreceğiz.

🔍 Mini Quiz

1. Decorator tek cümleyle nedir?
2. `@Component` bir sınıfa ne ekler?
3. Teknik olarak bir decorator nedir (ne tür bir yapı)?
4. Modern Angular'da `@Input()` yerine ne kullanılır?

🔑 Cevaplar

1. Bir sınıfa/alana ek bilgi (metadata) ve davranış iliştiiren özel bir işaret.
2. “Bu bir Angular bileşenidir” kimliğini ve yapılandırmasını (selector, template...).
3. Bir fonksiyon; üstüne yazıldığı şeyi alıp ona metadata ekler.
4. Signal tabanlı `input()` fonksiyonu (çıkı için `output()`).

❌ Anti-Patterns

- ❌ **Decorator’ları “sihir” sanmak.** Sadece metadata ekleyen fonksiyonlardır; Angular derleyicisi okur.
- ❌ **Yeni projede `@Input/@Output` ezberlemek.** Modern yol `input()` / `output()`; eskiyi sadece tanı.
- ❌ **Gereksiz yere kendi decorator’ını yazmaya çalışmak.** Başlangıçta neredeyse hiç gerekmez; Angular’ınkileri kullan.

✅ Bu Dersten Çıkarılacaklar

- ✓ Decorator, sınıfa/alana metadata ve davranış ekleyen `@` işaretli bir fonksiyondur.
- ✓ Angular’ın temel sınıf decorator’ları: `@Component`, `@Injectable`, `@Directive`, `@Pipe`.
- ✓ Decorator sınıfın içeriğini değiştirmez; ona “nasıl davranılacağını” söyler.
- ✓ Modern Angular alan decorator’larını (`@Input/@Output`) signal fonksiyonlarına taşıyor.
- ✓ Bu derste hedef: decorator’ları kavramsal anlamak — yazmak değil, okuyabilmek.



TypeScript, mevcut tiplerden **yeni tipler türetmek** için hazır jenerik yardımcılar sunar. Bunlara **utility types** denir. Aynı modeli elle tekrar yazmak yerine, tek bir interface'den onlarca varyasyon üretirsin. Angular'da formlar ve API'lerle çalışırken çok işine yarayacaklar.

Örneklerimiz şu modeli kullanacak:

TYPESCRIPT

```
interface Kullanici {  
  id: number;  
  ad: string;  
  email: string;  
  yas: number;  
}
```

Partial<T> — Tüm Alanlar Opsiyonel

Bir nesnenin sadece **bazı** alanlarını güncellerken kullanışlıdır:

TYPESCRIPT

```
function guncelle(id: number, degisiklikler: Partial<Kullanici>) {  
  // degisiklikler = { ad?: ..., email?: ..., yas?: ... }  
}  
  
guncelle(1, { email: "yeni@ornek.com" }); // sadece bir alan -> OK
```

Required<T> ve Readonly<T>

Required<T> tüm opsiyonel alanları **zorunlu** yapar. **Readonly<T>** tüm alanları **değiştirilemez** kılar:

TYPESCRIPT

```
const sabitKullanici: Readonly<Kullanici> = {  
  id: 1, ad: "Efsane", email: "e@x.com", yas: 28,  
};  
sabitKullanici.ad = "Emre"; // HATA: readonly
```

Pick<T, K> ve Omit<T, K>

Pick belirttiğin alanları **seçer**; **Omit** belirttiklerini **çıkartır**:

```
// Sadece id ve ad'dan oluşan tip
type KullaniciOzet = Pick<Kullanici, "id" | "ad">;
// { id: number; ad: string; }

// id hariç her şey (örn. "yeni kayıt" verisi – id'yi sunucu üretir)
type YeniKullanici = Omit<Kullanici, "id">;
// { ad: string; email: string; yas: number; }
```

🔗 Çok yaygın desen

Bir kayıt oluştururken `id` henüz yoktur (sunucu üretir). `Omit<Kullanici, 'id'>` ile tam da bu “id’siz oluşturma” tipini elde edersin. Aynı modeli ikinci kez yazmana gerek kalmaz.

Record<K, T> — Anahtar/Değer Haritası

Anahtarları ve değer tipini belirttiğin bir nesne tipi üretir:

```
type Durum = "yeni" | "islemde" | "bitti";

// Her durum için bir renk
const renkler: Record<Durum, string> = {
  yeni: "#1d6fe0",
  islemde: "#b8740a",
  bitti: "#1f8a4c",
};
```

Diğer Faydalılar

Utility	Ne yapar?
<code>ReturnType<F></code>	Bir fonksiyonun dönüş tipini çıkarır
<code>Parameters<F></code>	Bir fonksiyonun parametre tiplerini tuple olarak verir
<code>NonNullable<T></code>	<code>null</code> ve <code>undefined</code> ’i tipten çıkarır
<code>Exclude<T, U></code>	Bir union’dan belirli üyeleri çıkarır

Angular Bağlamında

Bir formu kısmen güncellemek (patch) ya da “oluşturma DTO’su” tanımlamak için:

```
// Servis: yeni görev oluşturunken id göndermeyiz
createGorev(veri: Omit<Gorev, "id">) { /* ... */ }

// Reactive form değerini kısmen yamamak
form.patchValue(parcaVeri as Partial<Gorev>);
```

❓ Mini Quiz

1. `Partial<T>` ne yapar?
2. Bir nesneden `id` alanını çıkaran tipi nasıl yazarsın?
3. `Record<Durum, string>` nasıl bir tip üretir?
4. Sadece `id` ve `ad` içeren bir tip için hangi utility kullanılır?

🔑 Cevaplar

1. Tipteki tüm alanları opsiyonel yapar.
2. `Omit<Kullanici, 'id'>`.
3. Anahtarları `Durum` üyeleri (yeni/islemde/bitti), değerleri `string` olan bir nesne tipi.
4. `Pick<Kullanici, 'id' | 'ad'>`.

⊗ Anti-Patterns

- ⊗ Aynı modelin varyasyonlarını elle tekrar yazmak. `Omit/Pick/Partial` ile türet; tek kaynaktan yönet.
- ⊗ `Partial` 'ı her yerde kullanmak. Gereksiz opsiyonellik, zorunlu alanların unutulmasına yol açar; sadece gerçekten kısmi veride kullan.
- ⊗ `Record` yerine gevşek `{ [key: string]: any }` yazmak. Anahtar ve değer tiplerini netleştir.

✅ Bu Dersten Çıkarılacaklar

- ✓ Utility types, mevcut tiplerden yeni tipler türetir; tekrarı önler.
- ✓ `Partial` (hepsi opsiyonel), `Required` (hepsi zorunlu), `Readonly` (değiştirilemez).
- ✓ `Pick` seçer, `Omit` çıkarır; `Omit<T, 'id'>` “oluşturma” tipleri için klasiktir.
- ✓ `Record<K, T>` anahtar/değer haritası tipi üretir.
- ✓ Angular’da form patch ve DTO tanımlarında çok işe yararlar.



Type Guards & Narrowing

Bir değişken birden fazla tip olabildiğinde (union), onu güvenli kullanmak için tipini **daraltmamız** gerekir. Buna **narrowing** denir; bunu yapan kontrollere de **type guard**. Angular'da özellikle HTTP cevapları ve opsiyonel değerlerde sürekli kullanacaksınız.

Narrowing Nedir?

TypeScript, yazdığınız koşullara bakarak tipi otomatik daraltır:

TYPESCRIPT

```
function bicimle(deger: string | number): string {  
  if (typeof deger === "number") {  
    // Bu blokta TypeScript deger'i "number" bilir  
    return deger.toFixed(2);  
  }  
  // Buraya geldiyse deger kesin "string"  
  return deger.trim();  
}
```

Narrowing Araçları

Araç	Ne için	Örnek
<code>typeof</code>	İlkel tipler (string/number/boolean)	<code>typeof x === 'string'</code>
<code>instanceof</code>	Sınıf örnekleri	<code>hata instanceof Error</code>
<code>in</code>	Nesnede alan var mı?	<code>'email' in nesne</code>
Eşitlik	Literal/union karşılaştırma	<code>durum === 'bitti'</code>
Truthiness	null/undefined/boş eleme	<code>if (deger) { ... }</code>

Custom Type Guard

Kendi kontrol fonksiyonunu yazıp TypeScript'e "bu doğruysa tip şudur" diyebilirsin. Sihirli kısım dönüş tipindeki `deger is Tip` ifadesidir:


```
interface Kedi { miyavla: () => void; }
interface Kopek { havla: () => void; }

// "deger is Kedi" => bu fonksiyon true dönerse, deger bir Kedi'dir
function kediMi(deger: Kedi | Kopek): deger is Kedi {
  return (deger as Kedi).miyavla !== undefined;
}

function sesCikar(hayvan: Kedi | Kopek) {
  if (kediMi(hayvan)) {
    hayvan.miyavla(); // burada tip: Kedi
  } else {
    hayvan.havla(); // burada tip: Kopek
  }
}
```

Discriminated Unions (Ayırt Edici Birleşimler)

Type guard'ların en temiz hâli. Her tipe ortak bir **etiket alanı** (genelde **kind** veya **type**) koyarsın; TypeScript bu alana bakarak tipi kusursuz daraltır:

```
type Sonuc =
  | { durum: "basari"; veri: string }
  | { durum: "hata"; mesaj: string };

function isle(s: Sonuc) {
  switch (s.durum) {
    case "basari":
      console.log(s.veri); // TypeScript: burada "veri" var
      break;
    case "hata":
      console.log(s.mesaj); // burada "mesaj" var
      break;
  }
}
```

Angular Bağlamında

Bir HTTP cevabı “yükleniyor / başarılı / hata” gibi durumlar taşıyabilir. Discriminated union + narrowing ile template ve mantıkta bunu güvenle yönetirsin. Ayrıca opsiyonel (?) alanlarda truthiness kontrolü **undefined** hatalarını engeller:

```
type YuklemeDurumu<T> =
  | { tip: "yukleniyor" }
  | { tip: "hazir"; veri: T }
  | { tip: "hata"; mesaj: string };

durum = signal<YuklemeDurumu<Kullanici[]>>({ tip: "yukleniyor" });
```

🔗 Mini Quiz

1. Narrowing tek cümleyle nedir?
2. Bir değerin `string` mi `number` mı olduğunu hangi operatörle ayırırsın?
3. Custom type guard'ın dönüş tipini özel yapan ifade nedir?
4. Discriminated union'da tipleri ayıran şey nedir?

🔗 Cevaplar

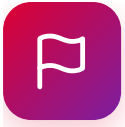
1. Bir union tipi, koşullara bakarak daha dar/kesin bir tipe indirgemek.
2. `typeof` (örn. `typeof x === 'number'`).
3. `deger is Tip` (type predicate).
4. Her üyede bulunan ortak bir etiket alanı (örn. `durum` / `kind`).

⊗ Anti-Patterns

- ⊗ `as` ile zorla tip dayatmak. `deger as Kullanici` kontrol yapmaz; yanlışsa runtime'da çöker. Önce narrow et.
- ⊗ Opsiyonel alanı kontrolsüz kullanmak. `user.adres.sehir` yerine önce `user.adres`'i kontrol et (truthiness / optional chaining).
- ⊗ Union'ları `any` 'ye kaçırmak. Narrowing tam da union'ları güvenle kullanman için var.

✅ Bu Dersten Çıkarılacaklar

- ✓ Union tipleri güvenle kullanmak için narrowing (daraltma) yaparız.
- ✓ Araçlar: `typeof`, `instanceof`, `in`, eşitlik, truthiness.
- ✓ Custom type guard: dönüş tipinde `deger is Tip` ile TypeScript'e tipi öğretirsin.
- ✓ Discriminated unions (ortak `kind` alanı) en temiz ve güvenli daraltma yöntemidir.
- ✓ Angular'da HTTP durumları ve opsiyonel değerler için günlük olarak kullanılır.



Öğrendiklerimizi tek bir küçük dosyada birleştirelim. Bu proje **saf TypeScript** (henüz Angular yok); amacı tipleri, generics'i, utility types'ı ve type guards'ı bir arada kullanmak. İstersen [typescriptlang.org/play](https://www.typescriptlang.org/play) (TS Playground) üzerinde tarayıcıda deneyebilirsin — kurulum gerekmez.

Hedef

Küçük bir **görev (task) modeli** ve onun etrafında tip-güvenli yardımcılar tasarlayacağız: model interface, durum union'ı, oluşturma DTO'su (Omit ile), generic bir sonuç sarmalayıcı ve bir type guard.

Adım 1 — Model ve Durum

TYPESCRIPT

```
interface Gorev {
  id: number;
  baslik: string;
  durum: GorevDurumu;
  oncelik: 1 | 2 | 3; // literal union: sadece 1, 2 veya 3
}

type GorevDurumu = "yeni" | "islemde" | "bitti";
```

Adım 2 — Oluşturma DTO'su (Utility Type)

Yeni görev oluştururken **id**'yi biz üretmeyiz; **Omit** ile çıkaralım:

TYPESCRIPT

```
type YeniGorev = Omit<Gorev, "id">;

let sonId = 0;
function gorevOlustur(veri: YeniGorev): Gorev {
  return { id: ++sonId, ...veri };
}
```

Adım 3 — Generic Sonuç Sarmalayıcı

İşlemlerin başarılı/başarısız sonucunu tip-güvenli taşıyalım (discriminated union + generic):

```

type Sonuc<T> =
  | { durum: "ok"; veri: T }
  | { durum: "hata"; mesaj: string };

function ilkAcikGorev(liste: Gorev[]): Sonuc<Gorev> {
  const acik = liste.find((g) => g.durum !== "bitti");
  return acik
    ? { durum: "ok", veri: acik }
    : { durum: "hata", mesaj: "Açık görev yok" };
}

```

Adım 4 — Type Guard

```

function bittiMi(g: Gorev): boolean {
  return g.durum === "bitti";
}

// Generic + utility birlikte: sadece bazı alanları güncelle
function guncelle(g: Gorev, degisiklik: Partial<Gorev>): Gorev {
  return { ...g, ...degisiklik };
}

```

Adım 5 — Hepsini Çalıştır

```

const g1 = gorevOlustur({ baslik: "Modül 2'yi bitir", durum: "islemde", oncelik: 1 });
const g2 = gorevOlustur({ baslik: "Mini projeyi dene", durum: "yeni", oncelik: 2 });

const sonuc = ilkAcikGorev([g1, g2]);

if (sonuc.durum === "ok") {
  // narrowing sayesinde burada sonuc.veri kesin Gorev
  const tamam = guncelle(sonuc.veri, { durum: "bitti" });
  console.log(tamam.baslik, "->", tamam.durum);
} else {
  console.log("Hata:", sonuc.mesaj);
}

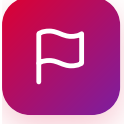
```

🔗 Kendi başına dene

Bu kodu çalıştır, sonra **kasıtlı hatalar** yap: `oncelik: 5` yaz (literal union reddedecek), `gorevOlustur` 'a `id` göndermeye çalış (Omit reddedecek), `sonuc.veri` 'yi `if` dışında kullan (narrowing olmadan hata). TypeScript'in seni nasıl koruduğunu canlı göreceksin.

✓ Bu Dersten Çıkarılacaklar

- ✓ Tek küçük dosyada: interface + literal union + `Omit` + generic `Sonuc<T>` + narrowing bir arada.
- ✓ `Omit<Gorev, 'id'>` ile “oluşturma” tipini modelden türettik.
- ✓ Discriminated union (`durum`) + `if` ile sonucu güvenle daralttık.
- ✓ Kasıtlı hatalarla TypeScript’in derleme-zamanı korumasını görmek en iyi öğrenme yoludur.



Harika — artık Angular’ı rahat yazacak TypeScript temeline sahipsin. Bu modülde dili “Angular gözüyle” gördük.

Öğrendiklerin

- **Tipler & interface:** TypeScript hataları derleme zamanında yakalar; nesne şekillerini interface ile tanımlarız (`readonly` , `extends`).
- **Generics:** Tipi parametre olarak almak (`<T>`); Angular’da `signal<T>` , `get<T>` her yerde.
- **Decorators:** Sınıfa metadata ekleyen `@` işaretleri; `@Component` , `@Injectable` Angular’ın temeli.
- **Utility types:** `Partial` , `Pick` , `Omit` , `Record` ile tipleri türetmek.
- **Type guards & narrowing:** Union’ları `typeof` / `in` /discriminated union ile güvenle daraltmak.

❓ Sırada ne var?

Modül 3’te artık gerçek koda giriyoruz: **İlk Angular Uygulaması, CLI & DevTools**. `ng new` ile oluşturulan proje yapısını satır satır çözecek, CLI komutlarını, `app.config.ts` ile bootstrapping’i, Angular DevTools ve v21’in MCP Server’ını göreceğiz; sonunda bir **Counter App** yapacağız.

💡 Kendini test et

Şu dört soruyu kendi cümlelerinle cevaplayabiliyor musun? “TypeScript hataları ne zaman yakalar? Generic ne işe yarar? `@Component` ne yapar? `Omit` ile `Pick` farkı ne?” Cevaplayabiliyorsan Modül 3’e hazırsın.

Takıldığın her yeri sor — hazır olduğunda Modül 3’e geçeriz. Kolay gelsin!